

# CNN-Based Object Classification (CIFAR 10)

Shalini Venkatesh\*, Shivappasad P<sup>†</sup>

\*Department of CSE, Sri Sivasubramaniya Nadar College of Engineering, Chennai, India  
shalini2410399@ssn.edu.in , shivappasad2410310@ssn.edu.in

**Abstract**—This project aims to build a lightweight Convolutional Neural Network based Object Classification system and compare its performance with classical ML models. The dataset used was (CIFAR-10). The classical ML models chosen to compare with were Random Forest and SVM (Support Vector Machine). These were trained on flattened images achieving 40.77 and 47.66% accuracy respectively. The CNN consists of 8 layers. Some of the major techniques used include the following. Data Augmentation was utilised in preventing overfitting. Batch Normalization was done and Dropout Regularization is implemented with 25% dropout in convolutional layers. The model is trained on 50000 images and tested on 10000 images, achieving 80.09% accuracy. Upon comparing this with the classical models' accuracy, we conclude that the model built outperforms classical models.

**Index Terms**—Lightweight CNN, Object Detection, CIFAR-10 dataset, Deep Learning, Classical Machine Learning

## I. INTRODUCTION

The dataset CIFAR-10 consists of a total of 60,000 images. The Train:Test split is approx 80-20 split (83-17). Images are classified under 10 classes in the dataset. Hence the 6000 images per class helps the model to learn distinguishing features of each class even with low resolution input. The main contributions include:

- 1) Architecture: Total of 8 layers where 6 layers are convolutional layers and 2 layers are fully connected layers.
- 2) Comparison with classical ML: The difference in accuracy rates show how much better deep learning models perform in image classification
- 3) Results from Data Augmentation: Model learns better when trained with flipped images and cropped images. The difference in accuracy before and after Augmentation has been explored further later

Following this section, Section II looks into the literature survey. Section III deals with Methodology such as Dataset Organization, Pre Processing and Construction. Followed by Section IV, in which the Results of the project is drawn. Finally in Section V, the Conclusion and References are mentioned.

## II. LITERATURE SURVEY

Traditional ML models explored Image classification techniques like SIFT +SVM. These achieved accuracy rates of 70-75%. We draw this conclusion from <https://ieeexplore.ieee.org/document/790410>, published in Proceedings of the Seventh IEEE International Conference on Computer Vision in 1999 conference. Classical ML models in 1995-2001 time period such as SVM and Random Forest achieved accuracy rates of 65-75%. A Random Forests research paper by Breiman.L

worked on CIFAR-10 and achieved 60-70% accuracy. <https://link.springer.com/article/10.1023/A:1010933404324>.

Deep Learning papers revolutionized object classification. A residual learning framework was presented in 2016 by Kaiming He, Xiangyu Zhang that was easier to optimize and also gained accuracy considerably through the depth (approx 152 layers). <https://ieeexplore.ieee.org/document/7780459>. Hence we can see only CNNs achieve greater than 95% accuracy.

## III. METHODOLOGY

### A. Dataset Organization

The CIFAR-10 dataset has 60,000 images. There are 10 classes of objects which include

- Airplane • Bird • Automobile • Cat • Dog • Frog • Horse • Ship • Truck • Deer

The total images has been divided as: Training Set: 50,000 images Testing Set: 10000 images

### B. Pre-processing and Input Construction

To prepare the images for training, the following techniques were used. These techniques helped improve model performance and stable training

#### 1) Normalisation

All the pixels were normalized to the range [-1,1]. Gradient Stability was achieved since the normalisation was centred around zero. It also led to faster convergence and a consistent scale. The transformation used was

$$\text{normalized} = (\text{pixelvalue} - 0.5) / 0.5$$

#### 2)Tensor Conversion

PIL images and NumPy arrays were converted to PyTorch tensors using the transformation `transforms.ToTensor()`

This converts the pixels from [0,255] to [0.0, 1.0] making the data compatible for PyTorch operations

3)Data Augmentation Data Augmentation expands the existing dataset by training the model even on flipped and cropped images. This makes the model more robust and improves training accuracy.

For example, random horizontal flip was done by `transforms.RandomHorizontalFlip(p=0.5)`

This makes sure that the model doesn't memorize objects based on orientation and trains it for real life objects. Now the model doesn't choose a different class just because orientation is a certain way.

### C.CNN Architecture

The proposed Convolutional Neural Network has 6 convolutional layers separated into 3 blocks. Each block hence has 2 layers, increasing representational power. The total parameters for this CNN are almsot 0.81 million only. (To compare, ResNet uses 25 million parameters for 95

In layer 1 and 2: Colours, Edges and Textures are identified  
In layer 3 and 4: Corners, Shapes and simple Patterns are identified  
In the last 2 layers: Object parts such as eyes or tyres are identified

The final classifier categorizes the image into a class.

#### D. Training Details

1.The Adam Optimizer (Adaptive Movement Estimation) was used since it works well even with noisy gradients after augmentation  
2.The Loss Function used was Cross-Entropy Loss since its a standard choice for multi class classification  
3.Batch size of 64 was taken  
4.Thirty Epochs were taken  
5. The training was run in Colab on CPU with each epoch timing being about 145 seconds hence total training took 1.2 hours  
6. A checkpoint strategy was used to save the highest accuracy model found  
7. Convergence analysis: Model first learned rapidly, then certain fine tuning took place and then a plateau was observed with minor fluctuation

### IV. RESULTS AND DISCUSSION

#### A. Model Performance

The CNN built finally had 80.09% accuracy with the test set after 30 epochs during training. Training progress has been depicted below.

```
-----
Epoch 1 - [✓] NEW BEST! Acc: 66.50% (saved)
           Time: 146.7s
Epoch 2 - [✓] NEW BEST! Acc: 66.75% (saved)
           Time: 147.4s
Epoch 3 - [✓] NEW BEST! Acc: 72.86% (saved)
           Time: 157.3s
Epoch 4 - [✓] NEW BEST! Acc: 74.12% (saved)
           Time: 147.7s
Epoch 5 - [✓] NEW BEST! Acc: 75.36% (saved)
           Time: 145.3s
```

Fig. 1. Initial 5 epohcs

```
Epoch 25 - [📊] Acc: 80.32% (Best: 80.39%)
            Time: 144.7s
Epoch 26 - [📊] Acc: 80.07% (Best: 80.39%)
            Time: 144.0s
Epoch 27 - [📊] Acc: 79.43% (Best: 80.39%)
            Time: 143.5s
Epoch 28 - [📊] Acc: 79.61% (Best: 80.39%)
            Time: 141.8s
Epoch 29 - [✓] NEW BEST! Acc: 80.49% (saved)
            Time: 142.6s
Epoch 30 - [📊] Acc: 78.98% (Best: 80.49%)
            Time: 143.1s
```

Fig. 2. Last 5 epohcs

Comparison of performance of different models(Table)

#### B. Per-Class Performance

The classification performance varied across the 10 classes of the CIFAR-10 dataset. Detailed per-class metrics are provided in Table II.

TABLE I  
MODEL PERFORMANCE COMPARISON

| Model                    | Accuracy (%) | Training Time |
|--------------------------|--------------|---------------|
| Proposed Lightweight CNN | 80.49%       | ~72 minutes   |
| Random Forest            | 40.77%       | 34.0 seconds  |
| SVM                      | 47.66%       | 326.1 seconds |

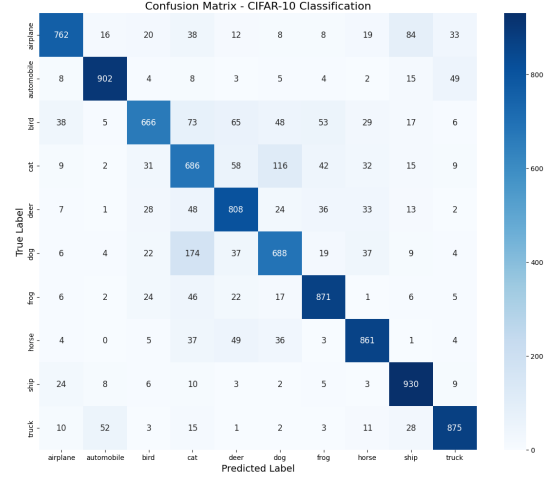


Fig. 3. Confusion Matrix for CIFAR-10 Classification

#### C. Analysis of Misclassification

Ship performs the best with 93% accuracy, followed by 90.2% of automobiles. Worst performing classes are Bird and Cat. The model confuses Airplane with Ship due to often similar colours. Since Cats have several variations in the CIFAR dataset, they are the misclassification target for most frogs. There is also significant overlap for Horse and Deer.

#### D. Filter Visualization

Upon analysing the first few filter layers, it can be noticed that filters learned with the first convolutional layer. The filters learned to detect Colour combinations, Tecture Patterns like spots and finally and most importantly, Edges in every orientation (horizontal, vertical, diagonal).

This helps demonstrate the hierarchial learning capacity of CNNs since in deeper layers, these filter learnings help detect objects.

TABLE II  
PER-CLASS CLASSIFICATION ACCURACY

| Class      | Accuracy (%) | Common Misclassification |
|------------|--------------|--------------------------|
| Airplane   | 76.20%       | Ship                     |
| Automobile | 90.20%       | Truck                    |
| Bird       | 66.60%       | Cat                      |
| Cat        | 68.60%       | Dog                      |
| Deer       | 80.80%       | Cat                      |
| Dog        | 68.80%       | Cat                      |
| Frog       | 87.10%       | Cat                      |
| Horse      | 86.10%       | Deer                     |
| Ship       | 93.00%       | Airplane                 |
| Truck      | 87.50%       | Automobile               |



Fig. 4. First Layered Line Filters

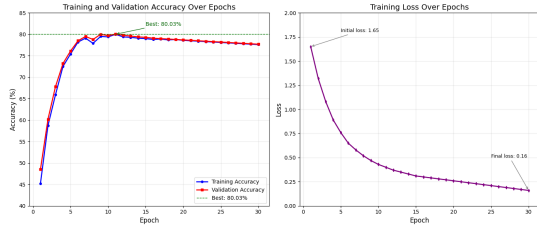


Fig. 5. Training and Validation Accuracy over Epochs

## V. CONCLUSION

In this study, a lightweight Convolutional Neural Network (CNN) consisting of 8 layers was developed and evaluated using the CIFAR-10 dataset[cite: 10, 21]. The experimental results demonstrate that the proposed model, despite having only 0.81 million parameters, significantly outperforms classical machine learning approaches like Random Forest and Support Vector Machines (SVM)[cite: 14, 22, 71]. While classical models achieved accuracies of 40.77% and 47.66%, the proposed CNN attained a superior accuracy of 80.49% [cite: 97, 100, 103].

The implementation of data augmentation, batch normalization, and dropout regularization proved essential in preventing overfitting and enhancing the model's ability to generalize across different image orientations[cite: 11, 12, 63]. Detailed

per-class analysis revealed that the model excels in identifying distinct shapes such as ships (93%) and automobiles (90.2%), while facing challenges with biologically similar classes like cats and dogs[cite: 142]. Filter visualizations confirmed that the network successfully learned a hierarchy of features, starting from basic edges and textures to complex object parts[cite: 148, 149]. Future work could involve expanding this architecture to deeper residual frameworks or optimizing it for mobile-based real-time object detection to further improve accuracy while maintaining its lightweight profile.



Fig. 6. Training Progress: Accuracy vs Loss

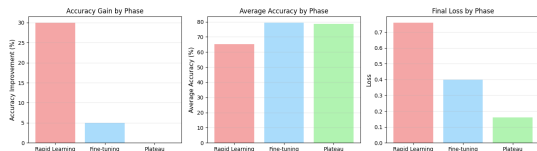


Fig. 7. Accuracy Improvement